

S02 - Max-Heap | Incident Response System

Implement a C application for managing an IT incident response system using a max-heap.

Data Structure

Define the **Incident** structure containing:

Field	Type	Description
id	int	Unique incident identifier
title	char*	Short description of the incident
affectedSystem	char*	e.g. "Database", "API Gateway", "Auth Service"
severity	int	Priority score from 1 to 10, used as the heap key
responseTime	int	Estimated response time in minutes

Requirements

1. Load at least 8 incidents from a .csv file and insert them one by one into a max-heap (array-based) ordered by severity. Print the heap array after loading. (2p)
2. Define an unsorted array of at least 5 incidents directly in code and build a max-heap from it using buildHeap. Print the resulting heap. (1p)
3. Implement the delete operation (deleteFromHeap). Using the heap from step 1, extract incidents one by one until only incidents with severity below a given threshold remain. Print each extracted incident and the heap array after each extraction. (2p)
4. Implement a function that iterates through the heap array and exports all incidents matching a condition into a dynamically allocated array of Incident* pointers, using a function pointer as the filter. Test with two functions - one that filters by severity above a threshold, and one that filters by a given affectedSystem name. Display the exported array after each call. (4p)

If the function pointer is not used and the condition is hardcoded, a maximum of 2p will be awarded for this step.

Step 4 - Example

```
// The filter function receives an Incident and returns 1 (match) or 0 (no match)
typedef int (*IncidentFilter)(Incident*);

int filterBySeverity(Incident* i) {
    return i->severity > 7;
}

int filterBySystem(Incident* i) {
    return strcmp(i->affectedSystem, "Database") == 0;
}

Incident** exportIncidents(Heap* heap, IncidentFilter filter, int* count);

// in main:
int count = 0;
Incident** result = exportIncidents(&heap, filterBySeverity, &count);
// print and free result
result = exportIncidents(&heap, filterBySystem, &count);
// print and free result
```

Provided Implementations

The following implementations are provided for convenience. You may use them as a starting point or implement your own.

Incident.c

```
#include <stdio.h>
#include "Incident.h"

void printIncident(const Incident* i) {
    printf("[%d] %s | %s | Severity: %d | Response: %d min\n",
        i->id, i->title, i->affectedSystem, i->severity, i->responseTime);
}
```

Heap.c - partial (provided functions)

```

void printHeap(Heap* heap) {
    for (int i = 0; i < heap->size; i++)
        printIncident(&heap->incidents[i]);
}

int loadIncidents(const char* filename, Heap* heap) {
    FILE* f = fopen(filename, "r");
    if (f == NULL) {
        printf("Error: could not open file '%s'\n", filename);
        return -1;
    }

    char line[256];
    int count = 0;

    while (fgets(line, sizeof(line), f) != NULL) {
        Incident i;

        char* token = strtok(line, ",");
        i.id = atoi(token);

        token = strtok(NULL, ",");
        i.title = malloc(strlen(token) + 1);
        strcpy(i.title, token);

        token = strtok(NULL, ",");
        i.affectedSystem = malloc(strlen(token) + 1);
        strcpy(i.affectedSystem, token);

        token = strtok(NULL, ",");
        i.severity = atoi(token);

        token = strtok(NULL, ",");
        token[strlen(token)] = '\0';
        i.responseTime = atoi(token);

        insertHeap(heap, i);
        count++;
    }

    fclose(f);
    return count;
}

void freeHeap(Heap* heap) {
    for (int i = 0; i < heap->size; i++) {
        free(heap->incidents[i].title);
        free(heap->incidents[i].affectedSystem);
    }
    free(heap->incidents);
    heap->incidents = NULL;
    heap->size = 0;
}

```

Example Data

incidents.csv (format: id,title,affectedSystem,severity,responseTime)

1	Database connection pool exhausted	Database	8	15
2	SSL certificate expired	API Gateway	6	30


```
3,Memory leak in auth service,Auth Service,9,10
4,DDoS attack detected,Load Balancer,10,5
5,Disk space critical,File Storage,7,20
6,Unauthorized login attempt,Identity Provider,8,12
7,Service timeout cascade,Microservices,9,8
8>Data breach detected,Customer Database,10,3
9,Config file corrupted,Deployment Service,5,45
10,Network partition,Message Queue,7,25
```

DO NOT TURN OFF OR RESTART THE COMPUTER DURING THE TEST

Notes

- Failure to free dynamically allocated memory will result in a deduction of 0.5p per requirement where memory leaks are present.
- Not respecting the specified Incident structure definition will result in a deduction of 1p.
- Incorrect implementation of the max-heap will result in a deduction of 1p per affected step.
- If the heap is not loaded from file, step 1 will not be graded.
- Code that does not compile will not be graded.
- Steps that generate runtime errors will not be graded.
- The use of AI tools is **not allowed** during the test.
- Your activity is being **monitored and logged** during the test.
- Write your full name as a comment at the top of main.c.